INDUSTRY PROBLEM TASK

Course Outcome Covered: CO 5

Assessment Tool: Industry Problem Task (BL4–BL6)
Weightage: 40%

Student Name : K Phani Sridhar Yogi
Register No.: 192311091

QUESTION 1

A hospital management system can be implemented as a SOAP-based web service that allows different departments such as reception, outpatient services, laboratory, pharmacy, and medical records to exchange patient information in a structured and interoperable manner. The service can expose operations such as registering a patient, scheduling an appointment, and retrieving medical records.

1.1 Proposed Service Operations

- addPatient(patient): Registers a new patient and returns a unique patient ID.
- getPatient(patientId): Retrieves basic patient information.
- scheduleAppointment(patientId, doctorId, dateTime): Creates an appointment and returns an appointment ID.
- getMedicalRecords(patientId): Retrieves the patient's authorized medical records.
- updatePatient(patient): Updates permitted patient details.

1.2 SOAP Message Structure

A SOAP message is an XML document consisting mainly of an Envelope, an optional Header, and a Body. The Envelope identifies the document as a SOAP message. The Header carries optional control information such as authentication, authorization, transaction, or security data. The Body contains the actual request or response. SOAP faults are returned in the Body when processing fails.

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
                  xmlns:hos="http://hospital.example.com/service">
    <soap:Header>
        <auth:Credentials xmlns:auth="http://hospital.example.com/auth">
            <auth:Token>secure-token</auth:Token>
        </auth:Credentials>
    </soap:Header>
    <soap:Body>
        <hos:addPatient>
            <hos:patient>
                <hos:name>Ravi Kumar</hos:name>
                <hos:dateOfBirth>1990-05-10</hos:dateOfBirth>
                <hos:department>Cardiology</hos:department>
            </hos:patient>
```

```
        </hos:addPatient>
    </soap:Body>
</soap:Envelope>
```

The server validates the SOAP envelope and namespace, authenticates the caller, processes the addPatient operation, stores the patient data, and returns a SOAP response.

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
                  xmlns:hos="http://hospital.example.com/service">
    <soap:Body>
        <hos:addPatientResponse>
            <hos:patientId>P10045</hos:patientId>
            <hos:status>SUCCESS</hos:status>
        </hos:addPatientResponse>
    </soap:Body>
</soap:Envelope>
```

1.3 Data Exchange Between Client and Server

- The hospital client creates an XML SOAP request according to the service contract.
- The request is sent over HTTP/HTTPS to the SOAP service endpoint.
- The server parses the XML, checks the SOAP namespace, validates the request data, and authenticates the caller.
- The requested business operation is executed against the hospital database.
- The server creates a SOAP response containing the result or a SOAP Fault if processing fails.
- The client validates and processes the response.

1.4 Justification for SOAP

SOAP is suitable for a hospital environment because it provides a standardized XML-based messaging framework and supports interoperability across different programming languages and platforms. SOAP headers can carry security and transaction information, while HTTPS can protect data during transmission. WS-Security can additionally support message-level authentication, integrity, and confidentiality. SOAP Faults provide a standard mechanism for reporting errors, and enterprise SOAP infrastructure can support reliable messaging, transactions, and auditing. These features are valuable when patient information must be exchanged consistently and securely between distributed hospital departments.

QUESTION 2

WSDL provides a machine-readable contract that tells external systems what operations are available, what data is exchanged, and how and where the service can be accessed. A WSDL document can therefore act as the interface contract between the enterprise application and its clients.

2.1 Key Components of WSDL

- Types: Defines the XML Schema data types used by the service, such as Customer, Account, or UpdateRequest.
- Message: Defines the input and output messages exchanged by each operation.

- PortType: Defines the abstract interface and lists the operations supported by the service.
- Binding: Specifies the concrete communication protocol and message format, such as SOAP over HTTP.
- Service: Provides the actual endpoint address where the service is available.

2.2 Sample WSDL Structure

```
<definitions name="EnterpriseDataService"
    targetNamespace="http://example.com/enterprise"
    xmlns:tns="http://example.com/enterprise"
    xmlns:xsd="http://www.w3.org/2001/XMLSchema"
    xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
    xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/">

    <types>
        <xsd:schema targetNamespace="http://example.com/enterprise">
            <xsd:element name="getDataRequest">
                <xsd:complexType>
                    <xsd:sequence>
                        <xsd:element name="id" type="xsd:string"/>
                    </xsd:sequence>
                </xsd:complexType>
            </xsd:element>
        </xsd:schema>
    </types>

    <message name="GetDataInput">
        <part name="parameters" element="tns:getDataRequest"/>
    </message>

    <message name="GetDataOutput">
        <part name="result" type="xsd:string"/>
    </message>

    <portType name="EnterpriseDataPortType">
        <operation name="getData">
            <input message="tns:GetDataInput"/>
            <output message="tns:GetDataOutput"/>
        </operation>
        <operation name="updateData">
            <input message="tns:UpdateDataInput"/>
            <output message="tns:UpdateDataOutput"/>
        </operation>
    </portType>

    <binding name="EnterpriseDataSoapBinding"
             type="tns:EnterpriseDataPortType">
        <soap:binding style="document"
                      transport="http://schemas.xmlsoap.org/soap/http"/>
    </binding>

    <service name="EnterpriseDataService">
        <port name="EnterpriseDataSoapPort"
              binding="tns:EnterpriseDataSoapBinding">
            <soap:address location="https://example.com/services/EnterpriseData"/>
        </port>
    </service>
</definitions>
```

2.3 How the WSDL Defines Integration

The types section establishes the structure of exchanged XML data. The message section identifies the individual input and output messages. The portType defines the logical interface independently of the transport protocol. The binding connects the abstract interface to SOAP and HTTP details. Finally, the service section gives the endpoint address. A client can use this contract to generate proxy classes or service stubs and communicate with the enterprise application without needing to know its internal implementation.

2.4 Importance of WSDL in Service Integration

WSDL improves interoperability because it provides a common and precise service contract. It reduces ambiguity between service providers and consumers, supports automated client generation, and separates the service interface from its implementation. When the contract is published through a service registry or development portal, external applications can discover and integrate with the service more easily.

QUESTION 3

When a SOAP request is not processed correctly or a response is delayed or incorrect, the problem can exist at several layers: the XML message, namespaces, SOAP action, authentication, endpoint, transport, server application, or backend database. A systematic debugging process should isolate each layer.

3.1 Possible Problems

- Malformed XML: Missing closing tags, invalid characters, incorrect nesting, or an invalid XML declaration.
- Incorrect SOAP envelope: The Envelope or Body uses an incorrect SOAP version or namespace.
- Namespace mismatch: The operation or elements use a namespace different from the one defined in the WSDL.
- Wrong operation or SOAPAction: The client invokes an operation name or action that the server does not expose.
- Invalid data types: Values do not conform to the XML Schema defined by the service contract.
- Incorrect endpoint: The client sends the request to an obsolete, incorrect, or unavailable URL.
- Authentication/security failure: Credentials, certificates, tokens, or WS-Security headers are missing or invalid.
- Server-side failure: Application logic, database access, timeout, or configuration problems may cause an incorrect or delayed response.

3.2 Debugging Approach

- Capture the complete request and response using a SOAP-aware testing/debugging tool or server logs.
- Validate the XML for well-formedness and compare the SOAP envelope and namespaces with the WSDL.

- Validate the message against the XML Schema so that required elements, data types, and ordering are correct.
- Confirm the SOAP version, operation name, SOAPAction, headers, and authentication information.
- Check that the endpoint URL, HTTP method, content type, TLS certificate, and network connectivity are correct.
- Inspect server logs for request parsing, authentication, application, database, and timeout errors.
- Test the operation with a known-good SOAP request to determine whether the fault is in the client or server.
- Verify the response against the expected WSDL/schema and check that the client maps the response correctly.
- Use controlled retries and appropriate timeout settings for transient failures, while avoiding duplicate processing of non-idempotent operations.

3.3 SOAP Fault and Error Handling

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
    <soap:Body>
        <soap:Fault>
            <faultcode>soap:Client</faultcode>
            <faultstring>Invalid patient identifier</faultstring>
        </soap:Fault>
    </soap:Body>
</soap:Envelope>
```

A SOAP Fault gives the client a standardized indication that processing failed. Client-side faults can identify invalid requests, while server-side faults can indicate unexpected processing failures. Sensitive internal details should not be exposed to external clients. Detailed diagnostics should remain in secured server logs, with correlation IDs used to trace a request across distributed components.

3.4 How the Corrected System Ensures Reliability

After correction, the client and server use the same WSDL-defined operations, namespaces, schemas, SOAP version, and endpoint configuration. XML and schema validation prevents malformed messages from reaching business logic. Standard SOAP Fault handling makes failures predictable, while logging and correlation IDs support diagnosis. HTTPS and suitable WS-Security mechanisms protect communication, and carefully designed timeouts, retries, and transaction controls reduce the impact of transient failures. Together these measures provide reliable and interoperable SOAP communication.

CONCLUSION

SOAP, WSDL, XML Schema, and related web-service standards provide a structured approach for building interoperable distributed applications. SOAP supplies the messaging framework, WSDL defines the service contract, and XML Schema ensures that exchanged data follows an agreed structure. Proper validation, security, logging, and error handling make the resulting services suitable for enterprise environments such as hospital management systems.